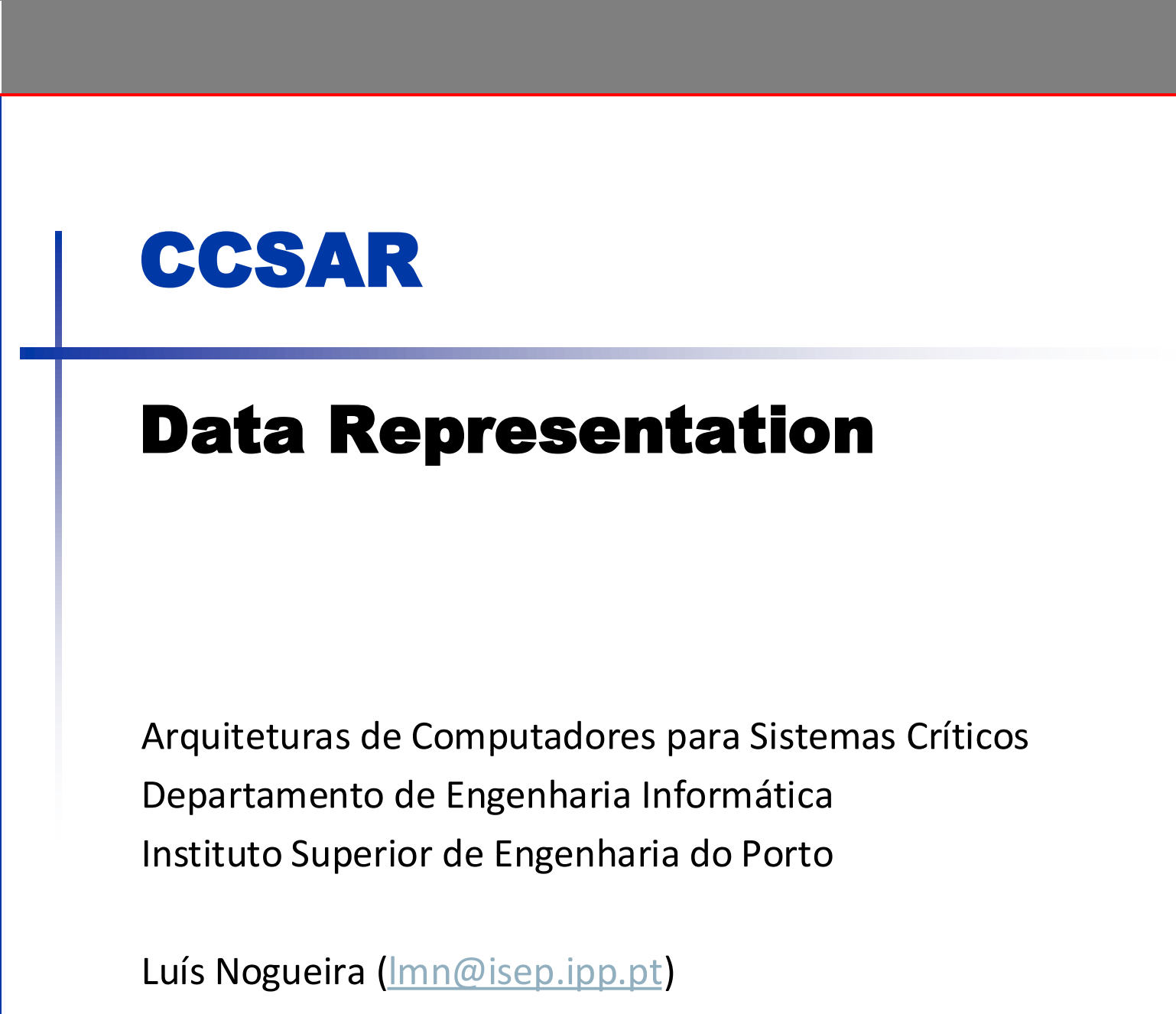




CCSAR

Data Representation

Arquiteturas de Computadores para Sistemas Críticos
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Introduction

- **This material revisits concepts from previous courses that will be assumed in the next lectures**
 - Use it to review concepts you are unfamiliar with or no longer remember

Topic	Why it matters
Binary and hexadecimal	Understanding machine representations
Two's complement	Signed integers and arithmetic
Bitwise operations	Low-level programming and hardware control
Shifts	Efficient low-level operations
Memory and addresses	Understanding pointers and memory
Pointers	Accessing and manipulating memory
Endianness	Understanding multi-byte data

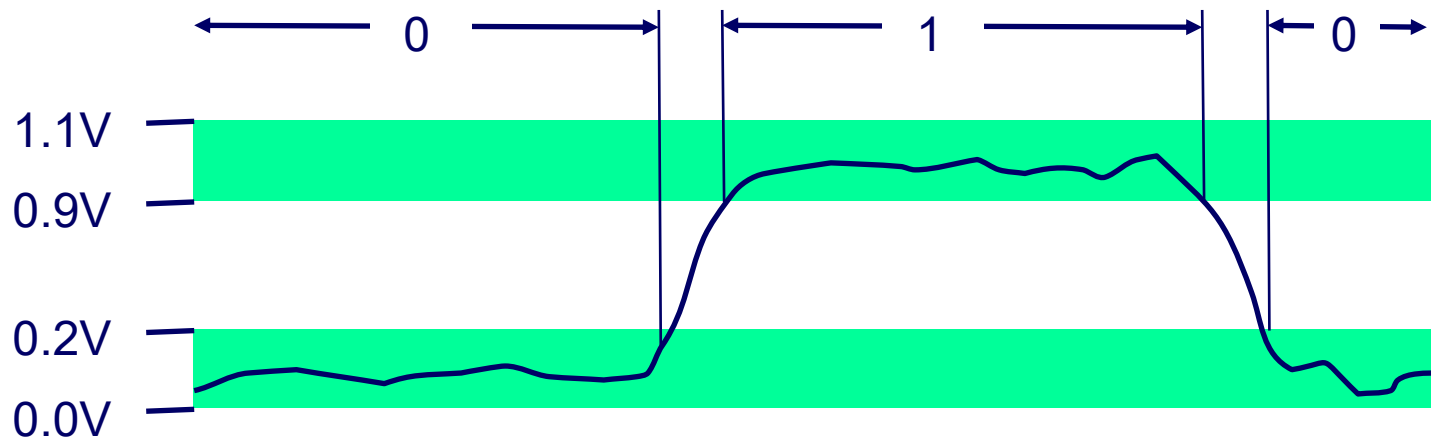
Everything is Bits

- **Binary digits (*bits*) form the basis of the digital revolution**

- Each bit is 0 or 1

- **Why bits?**

- Digital transistors operate in high and low voltage ranges
- Voltage range dictates binary value on wire
- Reliably transmitted on noisy and inaccurate wires



Everything is Bits

- In isolation, a single bit is not very useful but, by **encoding/interpreting** sets of bits in various ways
 - Computers determine what to do (instructions)
 - ... and represent and manipulate numbers, sets, strings, etc...
- Most computers use blocks of eight bits, or **bytes**, as the smallest addressable unit of memory
 - Common sizes: 1, 2, 4, 8, or 16 bytes

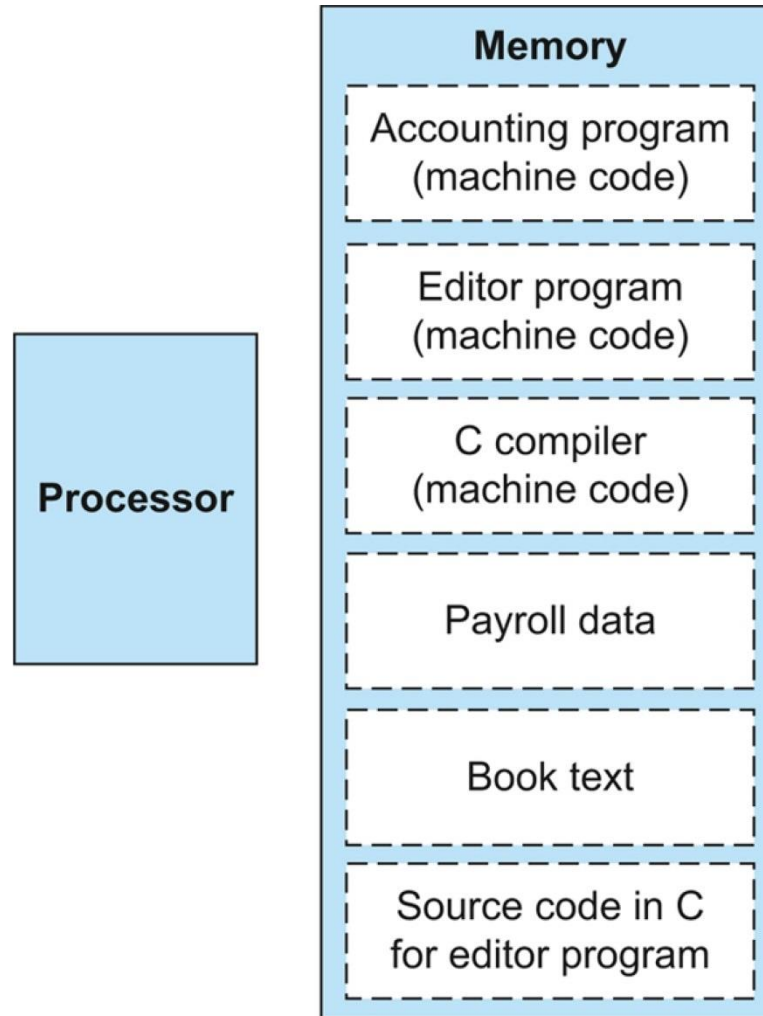
Encoding Byte Values

- A byte value can be **interpreted in many ways!**
- Depends upon how it's used
- For example, consider a byte with value $01010101_{(2)}$
 - As text: 'U'
 - As integer: $85_{(10)}$
 - As a IA32 instruction: `add %eax, %ecx`
 - As part of an address or real number
 - As a medium gray pixel in a gray-scale image
 - Could be interpreted in MANY other ways...

Stored Program Concept

- **Modern computers are built on two key principles:**
 1. Instructions are represented as numbers
 2. Programs are stored in memory to be read or written, just like data
- **These principles lead to the **stored program concept****
 - Instructions and data share the same memory and are both represented as binary values
 - Programs are shipped as files of binary numbers
 - Computers can inherit ready-made software provided they are compatible with an existing instruction set and operating system

Stored Program Concept



Using Bits to Represent Numbers

- Just like decimal except there are only two digits
 - 0 and 1
- Everything is based on **powers of 2** (1, 2, 4, 8, 16, ...)
 - Instead of powers of 10 (1, 10, 100, 1000, ...)
- Binary numbers are sums of powers of 2
 - $11011_{(2)} = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$
 $= 1 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1$
 $= 27_{(10)}$
 - $11101101101101_{(2)} = 15213_{(10)}$
 - $1.1101101101101_{(2)} \times 2^{13} = 1.5213 \times 10^4_{(10)}$

Using Bits to Represent Numbers

- **Binary: $00000000_{(2)}$ to $11111111_{(2)}$**
- **Decimal: $0_{(10)}$ to $255_{(10)}$**
- **Hexadecimal: $00_{(16)}$ to $FF_{(16)}$**
 - Base 16 number representation
 - Use characters '0' to '9' and 'A' to 'F'

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

15213: 0011 1011 0110 1101
 3 B 6 D

Unsigned Binary Integers

- **Given an n-bit number:**

$$X = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$$

- **Range: 0 to $+2^n - 1$**

- Using 32 bits

- 0 to +4 294 967 295

- **Example**

- 0000 0000 0000 0000 0000 0000 0000 1011₂
= 0 + ... + 1×2³ + 0×2² + 1×2¹ + 1×2⁰
= 0 + ... + 8 + 0 + 2 + 1 = 11₁₀

2s-Complement Signed Integers

- Given an n-bit number

$$X = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$$

- Range: -2^{n-1} to $+2^{n-1} - 1$
 - Using 32 bits
 - $-2\ 147\ 483\ 648$ to $+2\ 147\ 483\ 647$

- Example

- $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_2$
 $= -1 \times 2^{31} + 1 \times 2^{30} + \dots + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$
 $= -2,147,483,648 + 2,147,483,644 = -4_{10}$

2s-Complement Signed Integers

- **Most significant bit is a sign bit**
 - 1 for negative numbers
 - 0 for non-negative numbers
- **$-(-2^n - 1)$ can't be represented**
- **Non-negative numbers have the same unsigned and 2s-complement representation**
- **Some specific numbers**
 - 0: 0000 0000 ... 0000
 - -1: 1111 1111 ... 1111
 - Most-negative: 1000 0000 ... 0000
 - Most-positive: 0111 1111 ... 1111

2s-Complement Signed Integers

- **Complement and add 1**

- Complement means $1 \rightarrow 0, 0 \rightarrow 1$

$$x + \bar{x} = 1111 \dots 111_2 = -1$$

$$\bar{x} + 1 = -x$$

- **Example: negate +2**

- $+2 = 0000 \ 0000 \dots 0010_2$
- $-2 = 1111 \ 1111 \dots 1101_2 + 1$
 $= 1111 \ 1111 \dots 1110_2$

2s-Complement Signed Integers

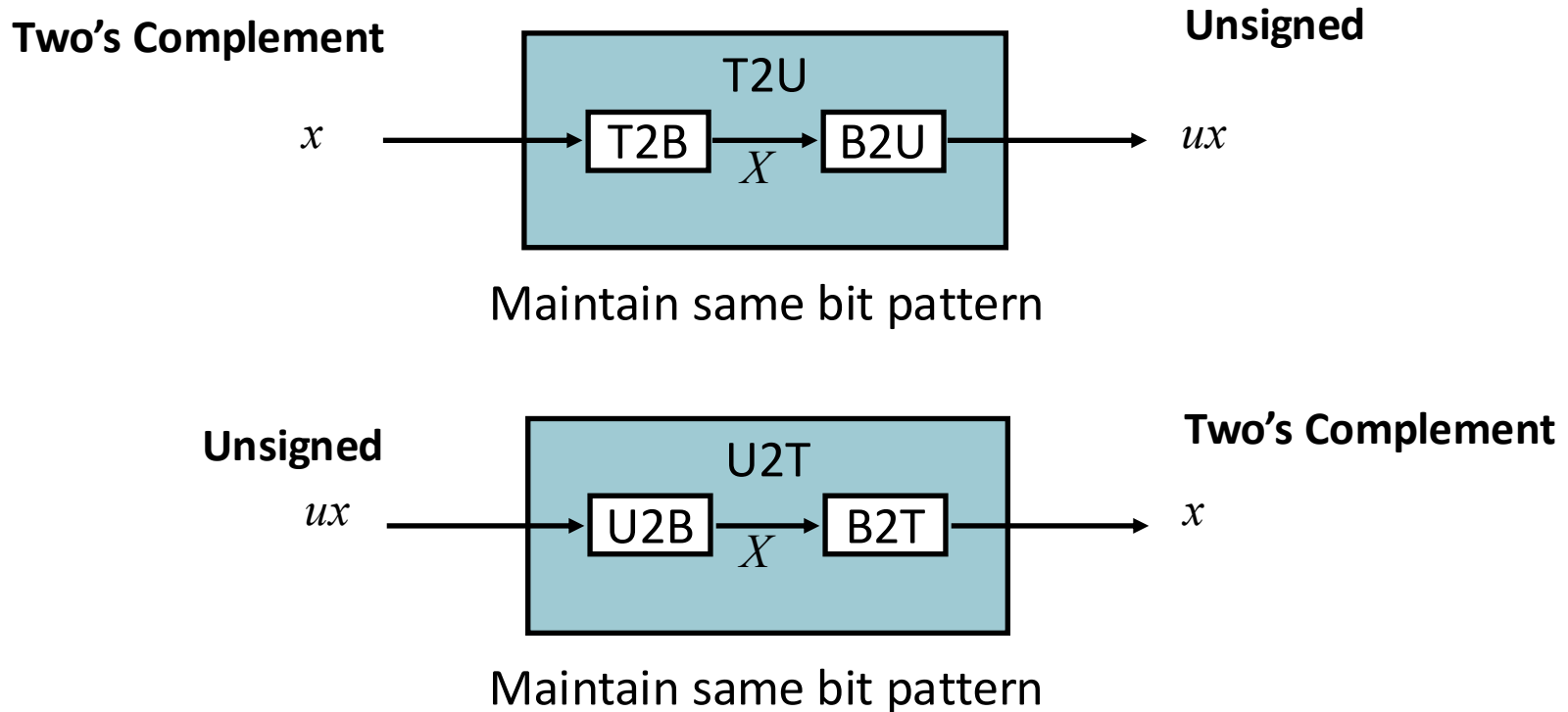
- **Using 2's complement number representation simplifies adder logic circuit design**
 - 0 is unique, easy to negate
 - Subtraction is adding, where one operand is negated (two's complement; to negate: flip the bits and add 1)
- **Overflow occurs when:**
 - A and B have the same sign
 - The result has a different sign
- **Can detect overflow by testing $C_{in} \neq C_{out}$ of the most significant bit**
 - which only occurs when previous statement is true

Data Representations

- Computers use a **limited number of bits** to encode a number
- Hence, some **operations can overflow/underflow** when the results are too large to be represented

C data type	RV32	RV64
char	1	1
short	2	2
int	4	4
long	4	8
long long	8	8
float	4	4
double	8	8
long double	10/12	10/16
pointer	4	8

Mapping Signed $\leftrightarrow$ Unsigned

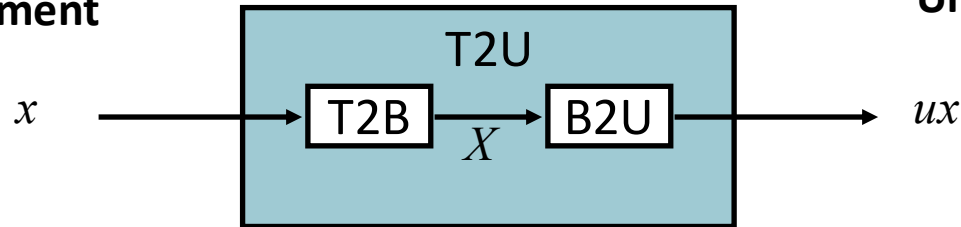


Mappings between unsigned and two's complement numbers:

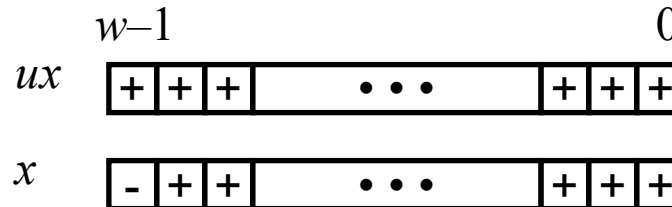
Keep bit representations and reinterpret

Relation Between Signed & Unsigned

Two's Complement



Maintain same bit pattern



Large negative weight
becomes
large positive weight

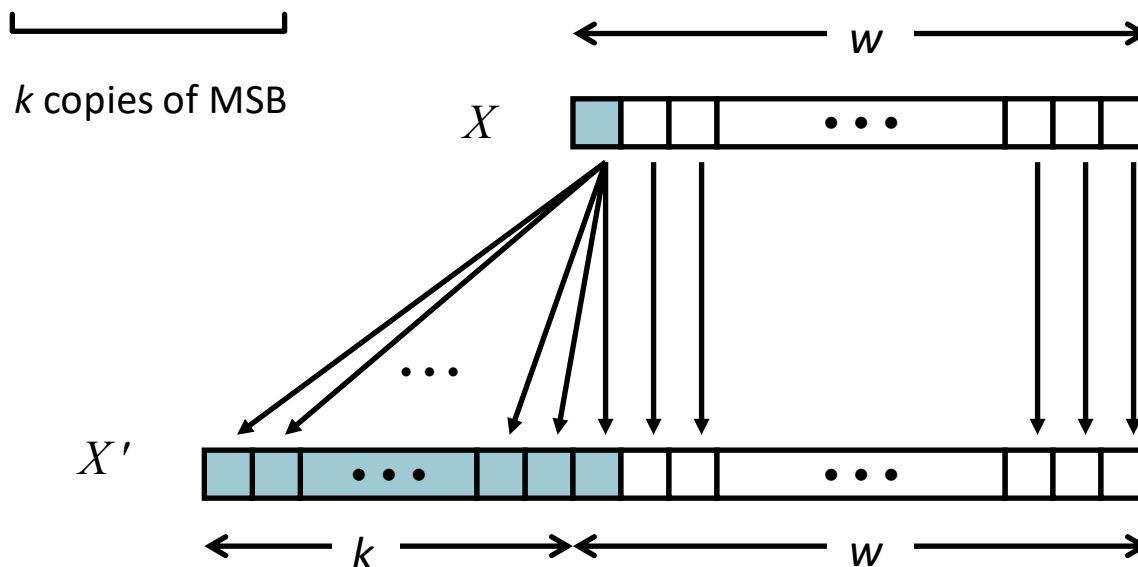
Expanding Bit Representation

■ Task:

- Given w -bit signed integer x
- Convert it to $(w+k)$ -bit integer **with same value**

■ Rule:

- Make k copies of sign bit:
- $X' = \underbrace{x_{w-1}, \dots, x_{w-1}}_{k \text{ copies of MSB}}, x_{w-1}, x_{w-2}, \dots, x_0$



Expanding Bit Representation

```
short x = 15213;
int  ix = (int)x;  /* expand to 32-bit value */

short y = -15213;
int  iy = (int)y;  /* expand to 32-bit value */
```

	Decimal	Hex	Binary
x	15213	3B 6D	00111011 01101101
ix	15213	00 00 3B 6D	00000000 00000000 00111011 01101101
y	-15213	C4 93	11000100 10010011
iy	-15213	FF FF C4 93	11111111 11111111 11000100 10010011

Truncating Numbers

■ Task:

- Given w -bit signed/unsigned integer x
- Convert it to k -bit integer ($k < w$), **possibly changing its value**

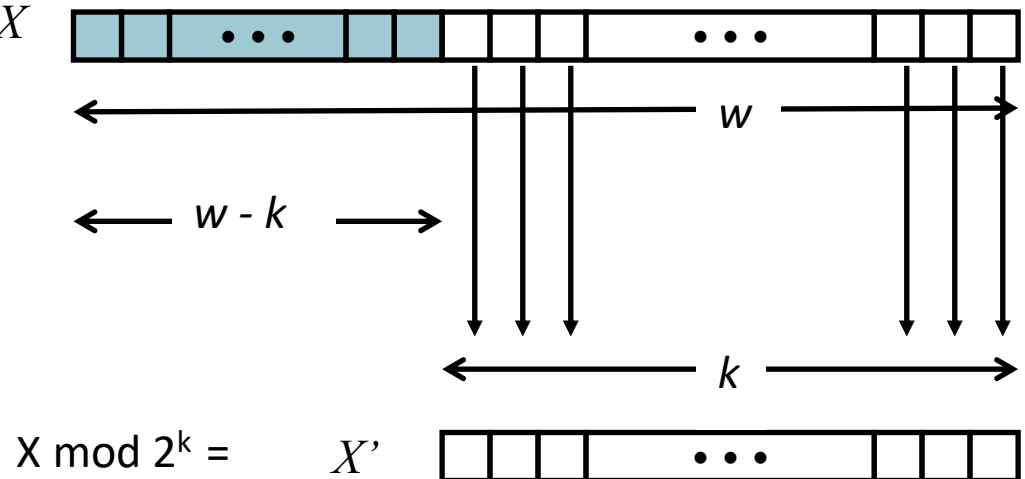
■ Rule:

- Drop the high-order $w - k$ bits:

- $X' = x_{k-1}, x_{k-2}, \dots, x_0$

$\underbrace{\hspace{10em}}$
 $w - k$ low-order bits

- **Reinterpret number**



Truncating Numbers

```
int    x    = 53191;
short  sx   = (short)x;      /* truncate to 16-bit value */
int    y    = sx;           /* expand to 32-bit value */
```

	Decimal	Hex	Binary
x	53191	00 00 CF C7	00000000 00000000 11001111 11000111
sx	-12345	CF C7	11001111 11000111
y	-12345	FF FF CF C7	11111111 11111111 11001111 11000111

Integer Arithmetic

- **Many beginning programmers are surprised to find that...**
 - Adding two positive numbers can yield a negative result
 - The comparison $x < y$ can yield a different result than the comparison $x - y < 0$
- **These properties are artifacts of the **finite nature of computer arithmetic****
 - Integer arithmetic performed by computers is really a form of **modular arithmetic**
- **Understanding the nuances of computer arithmetic can help programmers write more reliable code**

Arithmetic: Basic Rules

■ Addition

- True value requires **w+1 bits** in the worst case
- Unsigned/signed: Normal addition **followed by truncate and reinterpretation**
- Unsigned: addition mod 2^w
 - Mathematical addition + possible subtraction of 2^w
- Signed: modified addition mod 2^w (result in proper range)
 - Mathematical addition + possible addition or subtraction of 2^w

■ CPUs use the same machine instruction to perform either unsigned or signed addition/subtraction

- Same operation on bit level

Unsigned vs Signed Addition

- Same bits, different interpretation, different result

unsigned char

1110 1001	E9	223
+ 1101 0101	+ D5	+ 213
1 1011 1110	1BE	446
1011 1110	BE	190

char

1110 1001	E9	-23
+ 1101 0101	+ D5	+ -43
1 1011 1110	1BE	-66
1011 1110	BE	-66

Arithmetic: Basic Rules

■ Multiplication

- True value requires **$2*w$ bits** in the worst case
- Unsigned/signed: Normal multiplication **followed by truncate and reinterpretation**
- Unsigned: multiplication mod 2^w
- Signed: modified multiplication mod 2^w (result in proper range)

■ On most machines, integer multiplication and division are fairly slow

- Replace them by constant factors with combinations of shift and addition/subtraction operations

■ Separate instructions are provided in RV32 for signed and unsigned multiplication/division

Unsigned vs Signed Multiplication

- Same bits, different interpretation, different result

unsigned char

	1110 1001	E9	233
*	1101 0101	* D5	* 213
	<u>1100 0001 1101 1101</u>	<u>C1DD</u>	<u>49629</u>
	1101 1101	DD	221

char

	1110 1001	E9	-23
*	1101 0101	* D5	* -43
	<u>0000 0011 1101 1101</u>	<u>03DD</u>	<u>989</u>
	1101 1101	DD	-35

Takeaway

- Digital computers are implemented via logic circuits and thus **represent all numbers in binary** (base 2)
- The benefits of **two's complement integers** include simplicity and efficiency
 - Making them a popular choice for representing signed integers in computer architectures and programming languages
- We (humans) often write numbers as **decimal and hexadecimal** (programmers) for convenience
 - So, need to be able to convert to binary and back (to understand what the computer is doing!)
- Integer arithmetic performed by computers is really a form of **modular arithmetic**

Other Types of Data

- **Digital representation means that everything is represented only by numbers**
 - Text, code, sound, pictures, ...
- **For sound, pictures, other “real-world” values**
 - Make accurate measurements
 - Convert them to numeric values
- **The usual sequence:**
 - Data is converted into numbers by some mechanism
 - Numbers can be stored, retrieved, processed, transmitted
 - Numbers might be reconstituted into a version of the original

Using Bits to Represent Characters

- **Each character encoded in ASCII format**
 - American Standard Code for Information Interchange
 - Standard 7-bit encoding of character set
- **Each value between 0 and 127 represents a specific character**
- **Modern systems typically use Unicode, commonly encoded using UTF-8**
 - Which is backward compatible with ASCII and supports characters from virtually all languages

Using Bits to Represent Characters

	000	001	010	011	100	101	110	111
0000	NULL	DLE		0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EDT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

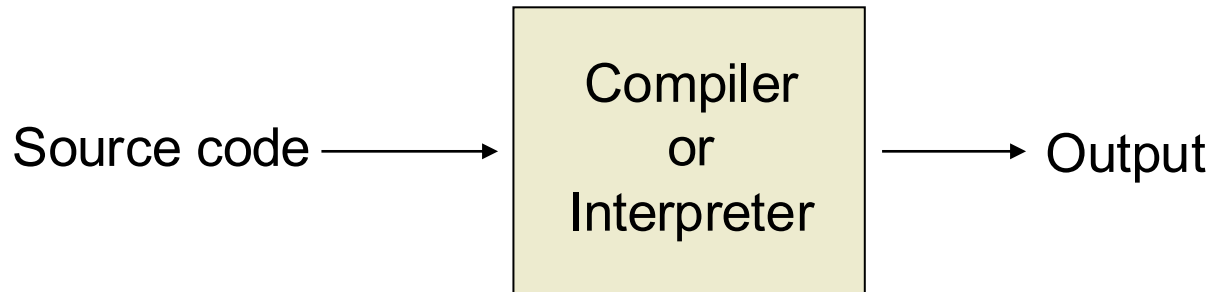
Using Bits to Represent Code

- **A program, from the perspective of the machine, is simply a sequence of bytes**
 - It has no information about the original source program (except some auxiliary tables maintained to aid in debugging)
- **Consider the following C function:**

```
int sum(int x, int y){  
    return x + y;  
}
```

Using Bits to Represent Code

- We depend on tools such as **compilation** and **interpretation** to get our written code into a form that the computer can execute
- **Both compilation and interpretation offer benefits and pitfalls**
 - Different execution models offer different trade-offs between performance, portability, startup time, and runtime optimization



Using Bits to Represent Code

- When compiled on a set of sample machines, we **generate machine code** having the following byte representations

Linux IA32:	55 89 e5 8b 45 0c 03 45 08 c9 c3
Windows IA32:	55 89 e5 8b 45 0c 03 45 08 5d c3
RV32:	00 b5 05 33 00 00 80 67
Linux x86-64:	55 48 89 e5 89 7d fc 89 75 f8 03 45 fc c9 c3

- Different machine types use different and incompatible instructions and encodings
 - Even identical processors running different OSes have differences in their coding conventions and hence are not binary compatible

Takeaway

- The abstraction of **data as numbers** is foundational to computer science
- This numeric representation allows computers to process, store, and transmit **diverse types of information efficiently**
- Code itself is represented as numbers, enabling computers to **interpret and execute** instructions efficiently

Boolean Algebra

- **Developed by George Boole in 19th Century**
 - Algebraic representation of logic
 - Encode “True” as 1 and “False” as 0

And

- $A \& B = 1$ when both $A=1$ and $B=1$

$\&$	0	1
0	0	0
1	0	1

Or

- $A | B = 1$ when either $A=1$ or $B=1$

$ $	0	1
0	0	1
1	1	1

Boolean Algebra

- **Developed by George Boole in 19th Century**
 - Algebraic representation of logic
 - Encode “True” as 1 and “False” as 0

Not

- $\sim A = 1$ when $A=0$

$\sim$	
0	1
1	0

Exclusive-Or (Xor)

- $A \wedge B = 1$ when either $A=1$ or $B=1$, but not both

$\wedge$	0	1
0	0	1
1	1	0

Extending Boolean Algebra

- Operate on bit vectors: operations applied **bitwise**

01101001	01101001	01101001	
& 01010101	01010101	^ 01010101	~ 01010101
<u> </u>	<u> </u>	<u> </u>	<u> </u>
01000001	01111101	00111100	10101010

- Bitwise operations have many properties in common with integer arithmetic
 - & → Intersection
 - | → Union
 - ^ → Symmetric difference
 - ~ → Complement

Bit-level Operations in C

- **Operations &, |, ~, ^ available in C**
 - Apply to any “integral” data type (signed and unsigned)
 - long, int, short, char, ...
 - View arguments as bit vectors
 - Arguments applied bit-wise

Examples (char data type)

$\sim 0x41_{(16)}$	$\rightarrow 0xBE_{(16)}$
$\sim 01000001_{(2)}$	$\rightarrow 10111110_{(2)}$
$\sim 0x00_{(16)}$	$\rightarrow 0xFF_{(16)}$
$\sim 00000000_{(2)}$	$\rightarrow 11111111_{(2)}$
$0x69_{(16)} \quad \& \quad 0x55_{(16)}$	$\rightarrow 0x41_{(16)}$
$01101001_{(2)} \quad \& \quad 01010101_{(2)}$	$\rightarrow 01000001_{(2)}$
$0x69_{(16)} \quad \quad 0x55_{(16)}$	$\rightarrow 0x7D_{(16)}$
$01101001_{(2)} \quad \quad 01010101_{(2)}$	$\rightarrow 01111101_{(2)}$

Contrast: Logic Operations in C

- **Contrast to logical operators &&, ||, !**
 - View 0 as “False”
 - Anything nonzero as “True”
 - Always return 0 or 1
 - **Early termination**

Examples (char data type)

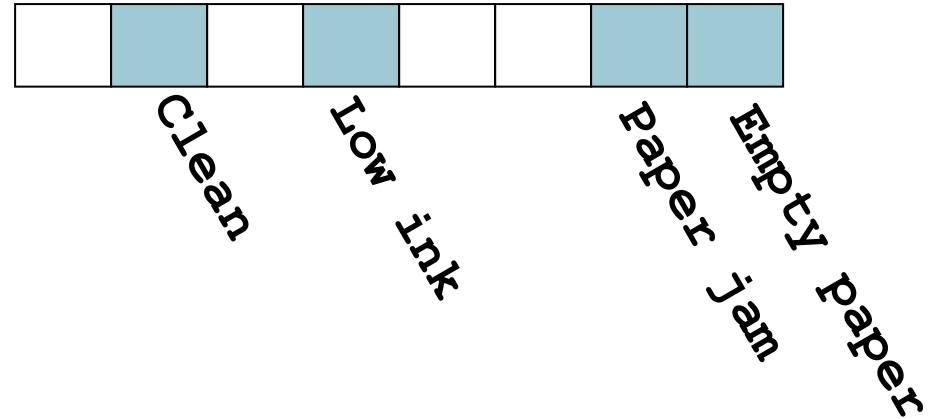
<code>!0x41</code>	<code>→ 0x00 (false)</code>
<code>!0x00</code>	<code>→ 0x01 (true)</code>
<code>!!0x41</code>	<code>→ 0x01 (true)</code>
<code>0x69 && 0x55</code>	<code>→ 0x01 (true)</code>
<code>0x69 0x55</code>	<code>→ 0x01 (true)</code>
<code>p && *p</code>	<code>(avoids null pointer access)</code>

Bit-level Operations

- **One common use of bit-level operations is to implement masking operations**
 - A *mask* is a bit pattern that indicates a selected set of bits within a word
- **Very useful in several practical applications**
 - IP addressing and subnetting
 - Hash tables
 - Controlling devices
 - Image processing
 - ...

Example: Printer Status Register

```
#define EMPTY    01
#define JAM      02
#define LOW_INK  16
#define CLEAN    64
```



```
char status;

if (status == (EMPTY | JAM)) ...;
if (status == EMPTY || status == JAM) ...;
while (!(status & LOW_INK)) ...;

status |= CLEAN; /* turns on CLEAN bit */
status &= ~JAM;  /* turns off JAM bit */
```

Shift Operations

- **C also provides a set of shift operations for shifting bit patterns to the left and to the right**
- **Arithmetic operators have precedence over shifts**
 - Getting the precedence wrong in C expressions is a common source of program errors, and often these are difficult to spot by inspection
- **For an n-bit value, the shift amount should be a value between 0 and $n - 1$**
 - **Undefined behavior** when shift amount < 0 or $\geq$ word size

Shift Operations

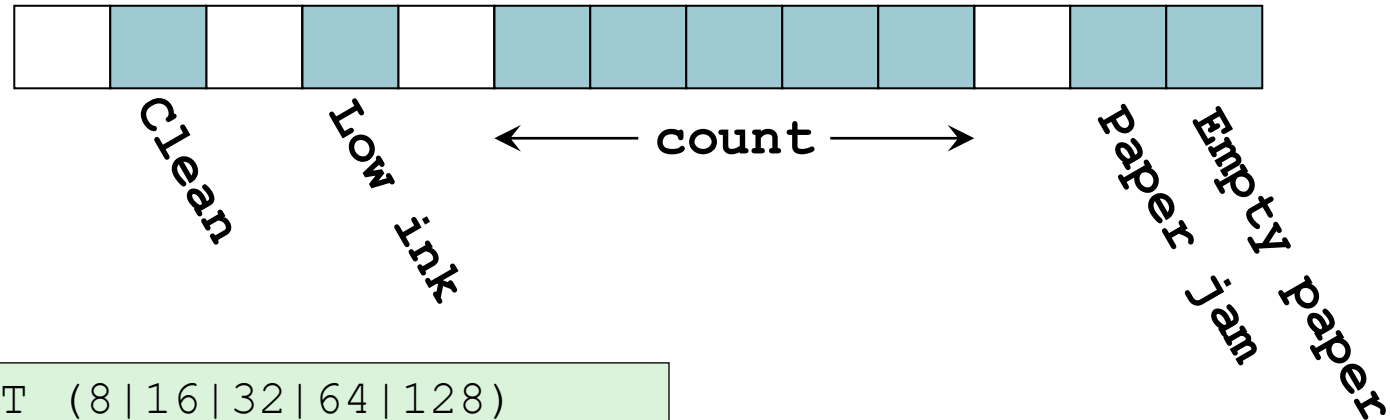
- **Left shift:** $x \ll y$
 - Shift bit-vector x left y positions
 - Throw away extra bits on left
 - **Fill with 0's on right**

Argument x	01100010
$\ll 3$	00010000
Log. $\gg 2$	00011000
Arith. $\gg 2$	00011000

- **Right shift:** $x \gg y$
 - Shift bit-vector x right y positions
 - Throw away extra bits on right
 - **Logical** shift
 - **Fill with 0's on left**
 - **Arithmetic** shift
 - **Replicate most significant bit on left**

Argument x	10100010
$\ll 3$	00010000
Log. $\gg 2$	00101000
Arith. $\gg 2$	11101000

Example: Printer Status Register



```
#define COUNT (8|16|32|64|128)
```

```
/* extract to c */
```

```
unsigned char c = (status & COUNT) >> 3;
```

```
/* insert v */
```

```
status = ((v << 3) & COUNT) | (status & ~COUNT);
```

Using Shifts for Multiplication/Division

■ Left shift ($x \ll k$)

- Unsigned/signed: multiplication by 2^k
- **Always logical shift**

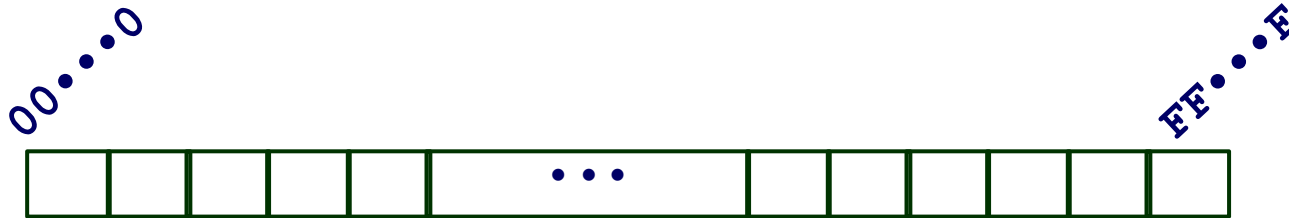
■ Right shift ($x \gg k$)

- Unsigned: logical shift, div (division + round to zero) by 2^k
- Signed: arithmetic shift
 - Positive numbers: div (division + round to zero) by 2^k
 - Negative numbers: div (division + round away from zero) by 2^k
 - Adding the bias $(1 \ll k) - 1$ before shifting corrects the rounding, so the final arithmetic shift matches truncation toward zero

Takeaway

- **Bitwise operations are extremely fast**
 - Enabling efficient computation, especially in low-level programming and performance-critical applications
- **Used to pack multiple pieces of data (flags, settings) into a single number**
 - Reducing memory usage and improving data processing efficiency
- **Allow precise control over individual bits**
 - Essential in hardware programming, networking, and systems where resources are limited

Byte-oriented Memory Organization



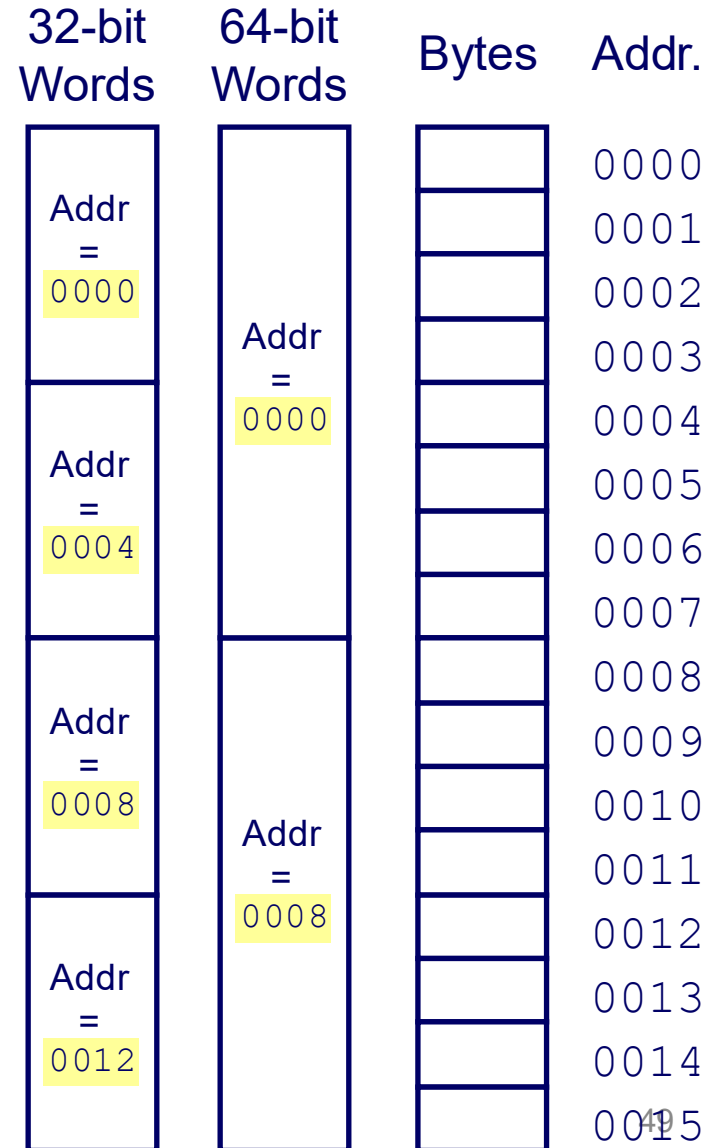
- **Programs refer to data *by address***
 - Conceptually, envision it as a very large array of bytes
 - In reality, it's not, but can think of it that way
 - An address is like an index into that array
- **The OS provides private address spaces to each process**
 - Think of a process as a program being executed
 - So, a program can clobber its own data, but not that of others

Machine Words

- **Any given computer has a “word size”**
 - Nominal size of integer-valued data and of addresses
- **Until recently, most machines used 32-bit word size**
 - Limits addresses to 4GB (2^{32} bytes)
- **Increasingly, machines have 64-bit word size**
 - Potentially 18.4 exabytes of addressable memory
- **Machines still support multiple data formats**
 - Fractions or multiples of word size
 - Always integral number of bytes

Word-oriented Memory Organization

- **Addresses specify byte locations**
 - Address of first byte in word
- **Addresses of successive words differ by 4 (32-bit) or 8 (64-bit)**



Pointers in C

- A pointer stores the **address** of an object in memory

```
int    b    = -15213;  
int    *p1  = &b;  
char   *p2  = (char*) &b;
```

- **The value of a pointer in C is the address of the first byte of some block of storage to which it points to**
 - Whether it points to an integer, a structure, or some other program object

Pointers in C

- **Pointer size is determined by the the full word size of the machine**
 - 4 bytes in 32-bit architectures, 8 bytes in 64-bit ones
- **Pointer type impacts pointer arithmetic and the number of bytes that are written/read to/from memory**
 - The computed value is scaled according to the size of the data type referenced by the pointer

```
int    b    = -15213;
int    *p1  = &b;           // p1+1 increments 4 bytes
char   *p2  = (char*) &b;   // p2+1 increments 1 byte
```

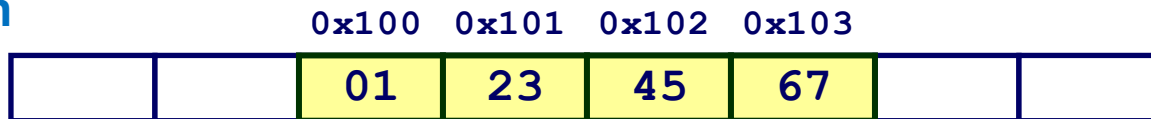
Byte Ordering

- **So, how are the bytes within a multi-byte word ordered in memory?**
- **Big Endian convention**
 - Least significant byte has highest address
 - Adopted by Sun, PPC Mac, Internet
- **Little Endian convention**
 - Least significant byte has lowest address
 - Adopted by x86, RISC-V, ARM processors running Android, iOS, and Windows

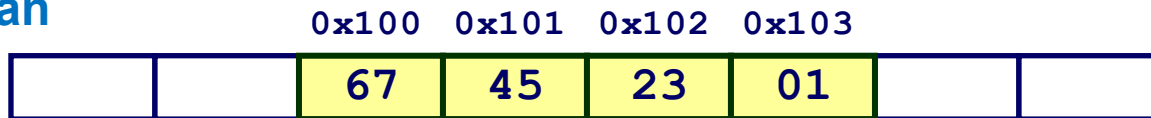
Byte Ordering: Example

- Assume `int x` has value of **0x01234567**...
- ... and the address given by `&x` is **0x100**

Big Endian



Little Endian



Examining Data Representations

- **Code to print byte representation of data**
 - Casting pointer to `unsigned char*` allows treatment as a byte array

```
void show_bytes(unsigned char* start, int len){
    int i;
    for (i = 0; i < len; i++)
        printf("%p\t0x%.2x\n", start+i, *(start+i));
    printf("\n");
}
```

printf directives:

%p: Print pointer

%x: Print hexadecimal

Example: show_bytes Execution

```
int a = 15213; /* 0x3B6D */  
show_bytes((unsigned char*) &a, sizeof(int));
```

Output (Linux/x86-64):

0x7fffb7f71dbc	6d
0x7fffb7f71dbd	3b
0x7fffb7f71dbe	00
0x7fffb7f71dbf	00

Takeaway

- **Pointers store the **memory address** of a variable, allowing efficient access and manipulation of data**
 - They enable dynamic memory management and are crucial for working with data structures like arrays, linked lists, and trees
- **Allow for **direct memory manipulation****
 - Leading to faster data access and efficient use of system resources, especially in low-level programming like system development and embedded systems
- **Understanding endianness is critical for **data portability** across different systems and architectures**
 - Especially in networking and file formats

Concluding Remarks

- Computers **encode information as bits**, generally organized as sequences of bytes
- Code itself is represented as numbers, enabling computers to **interpret and execute** instructions efficiently
- **Bitwise operations map naturally to efficient hardware operations**
 - Enabling efficient computation, especially in low-level programming and performance-critical applications

Concluding Remarks

- **A pointer holds the **memory address** of an object**
 - Enable indirection, dynamic memory and sharing/modifying data across functions
- **High-level languages are designed to accommodate a **wide range of word sizes and numeric encodings****
 - Understanding these encodings at the bit level is important for writing programs that operate correctly over the full range of numeric values
- **Different models of computers use **different conventions** for encoding numbers and for ordering the bytes within multi-byte data**